Chuyue（初悦）— AI 人生模拟器 完整技术文档

> **版本**：v1.0.0 | **日期**：2026-02-20 | **作者**：悠然若妃 | **原创作品，盗版必究**

目录

1. 项目概述
2. 技术栈与依赖
3. 系统架构
4. 目录结构
5. 数据库设计
6. 后端 API 接口
7. 前端状态管理
8. AI 引擎设计
9. 主题系统
10. 用户流程
11. 多剧本系统
12. 云端存档系统
13. 账号认证系统
14. 部署说明

1. 项目概述

Chuyue（初悦） 是一款以 AI 为核心驱动力的文字冒险人生模拟器。玩家通过创建角色、选择剧本世界观，与 AI 共同书写专属的命运故事。游戏采用「三选一 + 自由输入」的交互机制，每个回合 AI 自动生成三个差异化剧情选项，玩家也可以自由输入任意行动，由 AI 实时响应并推进故事。

本项目的核心设计理念是**沉浸感优先**：精心设计的三套国风主题（少女元氣、橘子汽水、高贵黑金）、流畅的粒子动画、细腻的 AI 叙事，共同构建出一个让玩家愿意反复回来的故事世界。

与市面上同类产品相比，Chuyue 的差异化优势体现在以下三个维度：

维度	Chuyue 特色

AI 推动机制	每 4 回合强制触发意外转折，每 7 回合引入新 NPC，避免剧情停滞
多剧本管理	玩家可创建最多 20 个独立剧本，随时切换世界观继续游戏
双重认证	支持 Manus OAuth 与独立游戏账号两套认证体系并行

---

2. 技术栈与依赖

2.1 核心框架

层级	技术	版本	用途
前端框架	React	19.2.1	UI 渲染与组件管理
后端框架	Express	4.21.2	HTTP 服务器
全栈类型安全	tRPC	11.6.0	前后端类型共享 RPC
数据库 ORM	Drizzle ORM	0.44.5	类型安全的数据库操作
数据库	MySQL / TiDB	—	持久化存储
样式框架	Tailwind CSS	4.x	原子化 CSS
UI 组件库	shadcn/ui + Radix UI	—	无障碍 UI 基础组件
状态序列化	superjson	1.13.3	Date 等类型安全传输
数据请求	TanStack Query	5.90.2	服务端状态管理

2.2 AI 与特殊功能

功能	技术/库	说明
AI 对话	Manus Built-in LLM / OpenAI 兼容 API	双模式：内置代理无需配置，外部 API 自定义
Markdown 渲染	streamdown 1.4.0	AI 输出的流式 Markdown 渲染

密码加密	bcryptjs 3.0.3	游戏账号密码哈希
JWT 认证	jose 6.1.0	游戏账号 Session 签发
动画	framer-motion 12.x	页面过渡与微交互
文件存储	AWS S3 SDK 3.x	云端文件存储

2.3 开发工具链

工具	版本	用途
TypeScript	5.9.3	全栈类型检查
Vite	—	前端构建与 HMR
tsx	—	服务端 TypeScript 执行
Vitest	2.1.9	单元测试框架
drizzle-kit	0.31.4	数据库迁移工具
pnpm	—	包管理器

---

3. 系统架构

3.1 整体架构图

Plain Text

```
┌──────────────────────────────────────────────────────────────────┐
│                        客户端（浏览器）                            │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  React 19 + Tailwind 4 + shadcn/ui                        │   │
│   │   ┌──────────┐   ┌──────────┐   ┌──────────────┐   │   │
│   │   │GameContext│  │tRPC Client│  │ aiEngine.ts  │   │   │
│   │   │(useReducer)│  │(TanStack) │  │(AI调用层)    │   │   │
│   │   └──────────┘   └──────────┘   └──────────────┘   │   │
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
                          │
                          │ HTTP / tRPC (JSON-RPC over HTTP)
┌─────────────────────────▼────────────────────────────────────────┐
```

```
│                    Express 服务器                                  │
│   ┌──────────────────────────────────────────────────────┐       │
│   │  tRPC Router                                          │       │
│   │  ├── auth (Manus OAuth)                               │       │
│   │  ├── account (游戏账号注册/登录)                      │       │
│   │  ├── game (存档/AI代理/API测试)                       │       │
│   │  ├── scripts (剧本管理)                               │       │
│   │  └── system (通知)                                    │       │
│   └──────────────────────────────────────────────────────┘       │
│   ┌──────────────────────────────────────────────────────┐       │
│   │  Drizzle ORM → MySQL/TiDB                             │       │
│   │  tables: users / game_saves / game_accounts /         │       │
│   │          game_scripts                                 │       │
│   └──────────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────────┘
                              │
┌─────────────────────────────▼────────────────────────────────────┐
│                 外部服务                                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐         │
│   │ Manus OAuth  │   │ Built-in LLM │   │  AWS S3      │         │
│   │ (用户认证)   │   │ (AI代理)     │   │ (文件存储)   │         │
│   └──────────────┘   └──────────────┘   └──────────────┘         │
└──────────────────────────────────────────────────────────────────┘
```

3.2 数据流说明

前端所有数据请求均通过 tRPC 客户端发往 `/api/trpc` 端点，服务器根据 procedure 名称路由到对应处理函数。AI 调用有两条路径：当用户开启「内置 AI 代理」时，前端调用 `trpc.game.aiProxy.mutate()`，由服务端使用 `BUILT_IN_FORGE_API_KEY` 调用 LLM；当用户配置了自己的 API Key 时，前端直接发起 `fetch` 请求到指定的 OpenAI 兼容端点。

---

4. 目录结构

Plain Text

```
life-sim/
├── client/                    # 前端代码
│   ├── public/                # 静态资源
│   └── src/
│       ├── _core/hooks/       # 核心 hooks (useAuth)
│       ├── components/
│       │   ├── game/          # 游戏核心组件 (16个)
│       │   │   ├── WelcomePage.tsx    # 欢迎首页
│       │   │   ├── LoginPage.tsx      # 游戏账号登录/注册
│       │   │   ├── ApiConfigPage.tsx  # API 配置页
```

```
│       │   │   ├── AppearancePage.tsx # 外观主题设置
│       │   │   ├── SetupPage.tsx      # 角色创建向导
│       │   │   ├── MainGame.tsx       # 主游戏容器
│       │   │   ├── StoryPage.tsx      # 故事/剧情页（核心）
│       │   │   ├── ScriptPage.tsx     # 剧本管理页
│       │   │   ├── SocialPage.tsx     # NPC 社交页
│       │   │   ├── InventoryPage.tsx  # 背包道具页
│       │   │   ├── NotesPage.tsx      # 记事本页
│       │   │   ├── SettingsPage.tsx   # 游戏设置页
│       │   │   ├── TopBar.tsx         # 顶部状态栏
│       │   │   ├── BottomNav.tsx      # 底部导航栏
│       │   │   ├── ModalManager.tsx   # 弹窗管理器
│       │   │   └── ThemeCanvas.tsx    # 主题粒子画布
│       │   └── ui/            # shadcn/ui 基础组件（40+个）
│       ├── contexts/
│       │   └── GameContext.tsx        # 全局游戏状态（核心）
│       ├── hooks/
│       │   └── useSaveGame.ts         # 云端存档 hook
│       ├── lib/
│       │   ├── aiEngine.ts            # AI 调用引擎（核心）
│       │   ├── i18n.ts                # 多语言支持
│       │   └── trpc.ts                # tRPC 客户端绑定
│       ├── pages/
│       │   └── Home.tsx               # 根页面（GameProvider 挂载点）
│       └── App.tsx                    # 路由配置
├── server/
│   ├── _core/                 # 框架核心（不建议修改）
│   │   ├── llm.ts             # LLM 调用封装
│   │   ├── oauth.ts           # Manus OAuth 处理
│   │   ├── context.ts         # tRPC 上下文
│   │   └── ...
│   ├── routers/
│   │   ├── game.ts            # 游戏存档 + AI 代理路由
│   │   ├── account.ts         # 游戏账号认证路由
│   │   └── scripts.ts         # 剧本管理路由
│   ├── routers.ts             # 主路由聚合
│   ├── db.ts                  # 数据库查询辅助函数
│   └── storage.ts             # S3 文件存储辅助函数
├── drizzle/
│   └── schema.ts              # 数据库表结构定义
├── shared/
│   ├── const.ts               # 前后端共享常量
│   └── types.ts               # 共享类型定义
├── server/scripts.test.ts     # 剧本路由单元测试
├── server/auth.logout.test.ts # 认证路由单元测试
├── todo.md                    # 项目任务追踪
└── package.json
```

5. 数据库设计

5.1 表结构总览

表名	用途	主要字段
`users`	Manus OAuth 用户	id, openId, name, email, role
`game_accounts`	游戏独立账号	id, username, passwordHash, salt, displayName
`game_saves`	游戏存档	id, userId, slotName, scriptId, playerData(JSON), round
`game_scripts`	剧本配置	id, userId, name, genre, worldSetting(JSON), isActive

5.2 users 表

SQL

```
CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  openId      VARCHAR(64) NOT NULL UNIQUE,   -- Manus OAuth 唯一标识
  name        TEXT,
  email       VARCHAR(320),
  loginMethod VARCHAR(64),
  role        ENUM('user','admin') DEFAULT 'user' NOT NULL,
  createdAt   TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt   TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL,
  lastSignedIn TIMESTAMP DEFAULT NOW() NOT NULL
);
```

5.3 game_accounts 表

SQL

```
CREATE TABLE game_accounts (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  username      VARCHAR(32) NOT NULL UNIQUE,  -- 2-20字符，字母/数字/中文
  password_hash VARCHAR(128) NOT NULL,         -- bcrypt 哈希
  salt          VARCHAR(32) NOT NULL,          -- 随机盐值
```

```
  display_name  VARCHAR(32),                    -- 可选显示名
  createdAt     TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt     TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL,
  lastSignedIn  TIMESTAMP DEFAULT NOW() NOT NULL
);
```

5.4 game_saves 表

SQL

```
CREATE TABLE game_saves (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  userId         INT NOT NULL,                  -- 关联 users.id 或 game_accounts
  slotName       VARCHAR(64) DEFAULT 'auto',    -- 存档槽名称
  scriptId       INT,                           -- 关联 game_scripts.id
  playerData     TEXT,                          -- 玩家角色 JSON
  npcsData       TEXT,                          -- NPC 列表 JSON
  storyData      TEXT,                          -- 故事历史 JSON（最近50条）
  inventoryData  TEXT,                          -- 背包 JSON
  notesData      TEXT,                          -- 记事本 JSON
  chatData       TEXT,                          -- 聊天记录 JSON
  apiConfigData  TEXT,                          -- API 配置 JSON（不含 key）
  appearanceData TEXT,                          -- 外观配置 JSON
  scriptData     TEXT,                          -- 剧本设定 JSON
  round          INT DEFAULT 0,
  gameYear       INT DEFAULT 2024,
  gameMonth      INT DEFAULT 1,
  gameDay        INT DEFAULT 1,
  createdAt      TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt      TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL
);
```

5.5 game_scripts 表

SQL

```
CREATE TABLE game_scripts (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  userId       INT NOT NULL,
  name         VARCHAR(64) NOT NULL,           -- 剧本名称
  description  TEXT,                           -- 剧本简介
  genre        VARCHAR(32) DEFAULT '现代',     -- 类型标签
  worldSetting TEXT,                           -- 世界观设定 JSON
  coverColor   VARCHAR(16) DEFAULT '#CBAF98',  -- 封面主色
  coverIcon    VARCHAR(8) DEFAULT '📖',        -- 封面图标
  isActive     BOOLEAN DEFAULT FALSE,          -- 是否为当前激活剧本
```

```
  totalRounds  INT DEFAULT 0,                    -- 累计游玩回合数
  lastPlayedAt TIMESTAMP DEFAULT NOW(),
  createdAt    TIMESTAMP DEFAULT NOW() NOT NULL,
  updatedAt    TIMESTAMP DEFAULT NOW() ON UPDATE NOW() NOT NULL
);
```

5.6 worldSetting JSON 结构

`game_scripts.worldSetting` 字段存储剧本世界观的 JSON 对象，结构如下：

JSON

```
{
  "era": "现代都市 2024年",
  "theme": "都市奋斗与情感",
  "rules": "现实世界规则，科技发达，社交媒体盛行",
  "special": "玩家可以通过努力改变命运，金钱与人脉是核心资源",
  "atmosphere": "现实主义，偶有温情与戏剧性转折"
}
```

---

6. 后端 API 接口

所有接口均通过 tRPC 暴露，前端使用 `trpc.*.useQuery()` 或 `trpc.*.useMutation()` 调用。

6.1 auth 路由

Procedure	类型	权限	说明
`auth.me`	query	public	获取当前 Manus OAuth 登录用户信息
`auth.logout`	mutation	public	清除 Session Cookie，登出

6.2 account 路由（游戏账号）

Procedure	类型	权限	说明
`account.register`	mutation	public	注册游戏账号（用户名+密码，bcrypt 加密）

`account.login`	mutation	public	登录游戏账号，签发 JWT Cookie
`account.logout`	mutation	public	清除游戏账号 Session
`account.getSession`	query	public	获取当前游戏账号登录状态
`account.checkUsernam e`	query	public	检查用户名是否已被占用

register 输入参数：

```TypeScript
{
  username: string,    // 2-20字符，支持中文/字母/数字/下划线
  password: string,    // 6-50字符
  displayName?: string // 可选显示名
}
```

register 返回值：

```TypeScript
{
  success: true,
  userId: number,
  username: string,
  displayName: string | null
}
```

6.3 game 路由

Procedure	类型	权限	说明
`game.saveGame`	mutation	protected	保存当前游戏进度到云端
`game.loadGame`	query	protected	读取指定存档（需校验所有权）
`game.getLatestSave`	query	protected	获取最新存档（用于「继续旅程」）

`game.listSaves`	query	protected	列出所有存档（最多10条）
`game.deleteSave`	mutation	protected	删除指定存档
`game.aiProxy`	mutation	public	后端 AI 代理（无需 API Key）
`game.testApiConnection`	mutation	public	测试外部 API 连通性

aiProxy 输入参数：

TypeScript

```
{
  messages: Array<{ role: "system" | "user" | "assistant", content: string }>,
  temperature?: number,   // 0-2，默认 0.85
  maxTokens?: number,     // 50-4000，默认 800
  schemaName?: string,    // JSON Schema 名称（结构化输出）
  schema?: Record<string, unknown>  // JSON Schema 定义
}
```

aiProxy 返回值：

TypeScript

```
{ success: true, content: string }
// 或
{ success: false, content: "", error: string }
```

6.4 scripts 路由

Procedure	类型	权限	说明
`scripts.getTemplates`	query	public	获取内置剧本模板列表（4套）
`scripts.getGenres`	query	public	获取剧本类型标签列表
`scripts.listScripts`	query	public	列出用户的所有剧本
`scripts.createScript`	mutation	public	创建新剧本（每用户上限20个）

`scripts.activateScript`	mutation	public	激活指定剧本（同时取消其他激活）
`scripts.updateScript`	mutation	public	更新剧本信息
`scripts.deleteScript`	mutation	public	删除剧本
`scripts.getScriptSaves`	query	public	获取指定剧本下的存档列表

内置剧本模板（4套）：

模板名	类型	封面	世界观主题
都市丽人	现代都市	🌆	现代都市奋斗与情感
古风仙缘	古风仙侠	🏯	修仙世界，灵气复苏
星际漂流	科幻未来	🚀	2387年，星际联邦时代
悬案迷踪	悬疑推理	🔍	现代都市，神秘案件

---

7. 前端状态管理

7.1 GameContext 架构

游戏的所有状态通过 `GameContext`（基于 `useReducer`）集中管理，避免 prop drilling 并保证状态一致性。

TypeScript

```
// 完整状态结构
interface GameState {
  onboardingStep: OnboardingStep;  // 当前流程步骤
  saveId: number | null;           // 云端存档 ID
  player: PlayerProfile | null;    // 玩家角色数据
  gameDate: GameDate;              // 游戏内时间
  round: number;                   // 当前回合数
  storyHistory: StoryMessage[];    // 故事历史（最近100条）
  isAiThinking: boolean;           // AI 思考状态
  aiStatusText: string;            // AI 状态文字
  npcs: NpcProfile[];              // NPC 列表
```

```
  activeChatNpcId: number | null;  // 当前聊天 NPC
  chatMessages: Record<number, ChatMessage[]>; // 聊天记录
  inventory: InventoryItem[];      // 背包物品
  notes: Note[];                   // 记事本
  apiConfig: ApiConfig;            // AI API 配置
  appearanceConfig: AppearanceConfig; // 外观主题配置
  currentPage: GamePage;           // 当前页面
  scriptData: Record<string, unknown>; // 当前剧本设定
  loggedInUsername: string | null; // 游戏账号用户名
  loggedInUserId: number | null;   // 游戏账号 ID
}
```

7.2 OnboardingStep 流程

玩家进入游戏后经历以下步骤：

Plain Text

```
welcome → login → api-config → appearance → char-setup → main
  欢迎页     登录页    API配置页     外观设置页     角色创建页    主游戏
```

每个步骤均可通过 `goToStep(step)` 跳转，`LOAD_SAVE` action 会直接跳至 `main` 步骤。

7.3 核心 Action 类型

Action	说明
`SET_ONBOARDING`	切换流程步骤
`SET_PLAYER`	设置玩家角色
`ADD_STORY`	添加一条故事消息
`SET_AI_THINKING`	设置 AI 思考状态
`LOAD_SAVE`	从存档恢复完整状态
`SET_SCRIPT_DATA`	更新当前剧本设定
`RESET_STORY`	重置故事历史（切换剧本时使用）
`SET_LOGGED_IN_USER`	设置游戏账号登录状态
`ADVANCE_ROUND`	回合数 +1

`UPDATE_PLAYER_STATS`	更新玩家属性值（带 0-100 范围限制）

7.4 PlayerProfile 数据结构

TypeScript

```
interface PlayerProfile {
  name: string;
  gender: "male" | "female" | "other";
  age: number;
  zodiac: string;           // 星座
  background: string;       // 背景故事
  avatarUrl: string;
  location: string;         // 所在城市
  weather: string;          // 当前天气
  height: number;           // 身高(cm)
  weight: number;           // 体重(kg)
  bust?: number; waist?: number; hips?: number; // 三围（可选）
  skinTone: string;         // 肤色描述
  stats: PlayerStats;       // 七维属性
  moneyCny: number;         // 人民币余额
  moneyUsd: number;         // 美元余额
  moneyCoins: number;       // 游戏金币
  traits: string[];         // 性格标签
  talents: string[];        // 才能标签
  isVirgin: boolean;
  hasFirstKiss: boolean;
  isPregnant: boolean;
  pregnancyDays: number;
}

interface PlayerStats {
  health: number;    // 健康值 0-100
  energy: number;    // 精力值 0-100
  beauty: number;    // 魅力值 0-100
  physique: number;  // 体质值 0-100
  mood: number;      // 心情值 0-100
  stress: number;    // 压力值 0-100
  iq: number;        // 智力值 0-100
}
```

---

8. AI 引擎设计

8.1 双模式 AI 调用

`aiEngine.ts` 实现了两套 LLM 调用路径，自动根据配置选择：

```Plain Text
用户配置了 API Key
    └── 直接 fetch → 用户指定的 OpenAI 兼容端点
用户未配置 / 开启内置代理
    └── fetch /api/trpc/game.aiProxy → 服务端 invokeLLM → Manus Built-in LLM
```

8.2 generateGameOpening（开场生成）

当玩家点击「开始游戏」时调用，生成 200-300 字的沉浸式开场白，并初始化 1-3 个 NPC。

输入： 玩家角色信息 + 剧本世界观设定 + API 配置

AI 输出 JSON 结构：

```JSON
{
  "opening": "开场叙事文字（200-300字）",
  "sceneTitle": "场景标题（4-8字）",
  "choices": [
    {"id": "c1", "text": "选项文字", "hint": "影响暗示", "type": "action"},
    {"id": "c2", "text": "选项文字", "hint": "影响暗示", "type": "explore"},
    {"id": "c3", "text": "选项文字", "hint": "影响暗示", "type": "dialogue"}
  ],
  "initialNpcs": [
    {
      "name": "NPC名字",
      "gender": "female",
      "age": 25,
      "occupation": "职业",
      "relationship": "关系",
      "background": "背景",
      "traits": ["性格1", "性格2"],
      "affection": 50
    }
  ]
}
```

8.3 advanceStory（剧情推进）

每次玩家做出选择后调用，推进故事并生成下一回合的三个选项。

AI 推动机制：

触发条件	效果
回合数 % 4 == 0	强制引入意外转折（偶遇/发现秘密/突发危机/意外机遇）
回合数 % 7 == 0	引入新 NPC 角色
其他回合	正常剧情推进

AI 输出 JSON 结构：

JSON

```
{
  "sceneTitle": "场景标题（可选）",
  "narrative": "故事叙事（150-250字，第二人称）",
  "choices": [
    {"id": "c1", "text": "行动选项", "hint": "影响暗示", "statHint": "+心情", "type
    {"id": "c2", "text": "对话选项", "hint": "影响暗示", "statHint": "-精力", "type
    {"id": "c3", "text": "休息选项", "hint": "影响暗示", "statHint": "+精力", "type
  ],
  "statChanges": {
    "health": 0, "energy": -5, "mood": 10, "stress": -5,
    "beauty": 0, "physique": 0, "iq": 0, "moneyCny": 0
  },
  "timeAdvance": 2,
  "mood": "场景氛围关键词",
  "event": "特殊事件标签或null",
  "newNpc": null
}
```

8.4 选项类型系统

类型	标签	说明
`action`	行动	主动出击类选项
`dialogue`	对话	与人交流类选项
`explore`	探索	调查/发现类选项
`rest`	休息	恢复精力类选项

special	特殊	特殊事件/隐藏选项

8.5 System Prompt 构建

buildSystemPrompt() 函数根据以下信息动态构建 System Prompt：

- 玩家角色的完整人设（姓名/年龄/性格/背景/属性）
- 剧本世界观设定（时代/主题/规则/特殊设定/氛围）
- 输出语言（支持简体中文/繁体中文/英文/日文/韩文）
- R18 模式开关（影响内容尺度）

---

9. 主题系统

9.1 三套主题

主题名	标识符	主色调	风格
少女元氣	sakura-girl	桃粉 #E6A6AC + 翠绿 #A1C2B1	清新少女，花瓣粒子
橘子汽水	orange-soda	质感灰 #454545 + 活力橙 #FF6000	现代活力，气泡粒子
高贵黑金	noble-black-gold	深邃黑 #111A19 + 典雅金 #BB6830	古典奢华，金粉粒子

9.2 CSS 变量体系

每套主题通过 .theme-* class 覆盖以下 CSS 变量：

CSS

```
--theme-bg       /* 主背景色 */
--theme-bg2      /* 次背景色 */
--theme-card     /* 卡片背景 */
--theme-primary  /* 主色调 */
--theme-accent1  /* 强调色1 */
--theme-accent2  /* 强调色2 */
--theme-text1    /* 主文字色 */
--theme-text2    /* 次文字色 */
```

```
--theme-text3    /* 辅助文字色 */
--theme-border   /* 边框色 */
--theme-shadow   /* 阴影色 */
```

9.3 字体系统

字体标识	字体族	风格
`cinzel-elegant`	Cinzel + Noto Serif TC	古典优雅
`cormorant-romantic`	Cormorant Garamond + Noto Serif TC	浪漫文学
`playfair-classic`	Playfair Display + Noto Serif TC	经典衬线
`noto-minimal`	Noto Sans TC	现代简约

---

10. 用户流程

10.1 新用户完整流程

Plain Text

```
1. 访问首页（WelcomePage）
   ├── 已登录 → 显示「继续旅程」按钮（读取最新存档）
   └── 未登录 → 显示「开始你的故事」按钮

2. 登录/注册（LoginPage）
   ├── 游戏账号注册：输入用户名+密码 → 服务端 bcrypt 加密 → 写入 game_accounts
   ├── 游戏账号登录：验证密码 → 签发 JWT Cookie
   └── Manus OAuth：跳转 OAuth 流程

3. API 配置（ApiConfigPage）
   ├── 开启内置代理（默认）→ 无需配置，直接下一步
   └── 自定义 API：填写端点/Key/模型 → 测试连通性

4. 外观设置（AppearancePage）
   └── 选择主题/字体/语言/UI 密度

5. 角色创建（SetupPage，6步向导）
   ├── 基础信息（姓名/性别/年龄/星座）
   ├── 外貌设定（身高/体重/肤色/三围）
```

```
    ├── 背景故事（出生地/背景/位置/天气）
    ├── 性格才能（性格标签/才能标签）
    ├── 初始 NPC（关系人设定）
    └── 确认创建

6. 主游戏（MainGame）
    ├── 点击「开始游戏」→ AI 生成开场剧情
    ├── 阅读故事 → 选择三选一选项或自由输入
    ├── AI 推进故事 → 更新属性 → 自动存档
    └── 底部导航：故事/社交/剧本/背包/设置
```

---

11. 多剧本系统

11.1 设计理念

多剧本系统允许玩家在同一账号下维护多个平行故事线。每个剧本有独立的世界观设定，切换剧本时故事历史会重置，但玩家角色数据保留。

11.2 ScriptPage 功能

ScriptPage 提供三种视图模式：

视图	功能
`list`	显示所有剧本卡片，支持激活/删除/查看详情
`create`	创建新剧本（模板选择或自定义）
`detail`	查看剧本详情（世界观/游玩记录/关联存档）

11.3 剧本切换流程

Plain Text

```
用户点击「使用此剧本」
    ↓
activateScript mutation（更新数据库 isActive 字段）
    ↓
dispatch SET_SCRIPT_DATA（将 worldSetting 注入 GameContext）
    ↓
dispatch RESET_STORY（清空故事历史，round 重置为 1）
    ↓
dispatch SET_ONBOARDING("main")（进入主游戏）
```

```
    ↓
StoryPage 检测到 storyHistory 为空 → 显示「开始游戏」按钮
    ↓
玩家点击「开始游戏」→ generateGameOpening（携带新剧本世界观）
```

---

12. 云端存档系统

12.1 自动存档机制

每次 AI 推进故事后（`advanceStory` 完成），`StoryPage` 会自动调用 `useSaveGame().saveGame("auto")` 进行静默存档。存档使用防重入锁（`isSavingRef`）避免并发写入。

12.2 存档数据序列化

所有复杂对象（PlayerProfile、NpcProfile[]、StoryMessage[] 等）均序列化为 JSON 字符串存入 TEXT 字段。读取时反序列化，并对 `Date` 类型进行特殊处理（`new Date(m.timestamp)`）。

12.3 存档隔离

`game.saveGame` 和 `game.loadGame` 均为 `protectedProcedure`，通过 `ctx.user.id` 确保用户只能访问自己的存档。删除存档时额外校验 `save.userId === ctx.user.id`。

---

13. 账号认证系统

13.1 双认证体系

认证方式	实现	Cookie 名	适用场景
Manus OAuth	`server/_core/oauth.ts`	`manus_session`	已有 Manus 账号的用户
游戏账号	`server/routers/account.ts`	`game_session`	不想绑定 Manus 的玩家

13.2 游戏账号安全设计

密码存储采用 bcrypt 算法（cost factor 12），每个账号生成独立的随机盐值。用户名验证规则：2-20 字符，支持中文/英文字母/数字/下划线，禁止纯数字用户名（防止与自动生成的随机数字 ID 混淆）。

若用户注册时未设置用户名，系统自动生成格式为 `user_XXXXXX` 的随机数字 ID（6位随机数），确保唯一性。

13.3 Session 管理

游戏账号登录后签发 JWT，存入 HttpOnly Cookie。`account.getSession` 接口在每次页面加载时验证 Cookie 有效性，恢复登录状态到 `GameContext` 。

---

14. 部署说明

14.1 环境变量

变量名	说明	必须
`DATABASE_URL`	MySQL/TiDB 连接字符串	✓
`JWT_SECRET`	Session Cookie 签名密钥	✓
`BUILT_IN_FORGE_API_KEY`	Manus 内置 LLM API Key	✓
`BUILT_IN_FORGE_API_URL`	Manus 内置 LLM 端点	✓
`VITE_APP_ID`	Manus OAuth 应用 ID	✓
`OAUTH_SERVER_URL`	Manus OAuth 后端地址	✓
`VITE_OAUTH_PORTAL_URL`	Manus 登录门户地址	✓
`OWNER_OPEN_ID`	项目所有者 Open ID	—
`OWNER_NAME`	项目所有者名称	—

14.2 启动命令

Bash

```bash
# 安装依赖
pnpm install

# 推送数据库 schema
pnpm db:push

# 开发模式
```

```
pnpm dev

# 生产构建
pnpm build

# 运行测试
pnpm test
```

14.3 测试覆盖

测试文件	测试数	覆盖范围
`server/scripts.test.ts`	6	剧本模板/类型/路由结构
`server/auth.logout.test.ts`	1	登出 Cookie 清除

文档版本：v1.0.0 | 最后更新：2026-02-20 | 原创作品，版权归悠然若妃所有